

Hevo Data · CSE Technical Assessment

End-to-End Pipeline: PostgreSQL → Hevo → Snowflake → dbt

Arpan Banerjee | April 2026 | github.com/JBLarpan/hevo-dbt-assessment

1. Approach

Built a real-time ELT pipeline to replicate e-commerce data (customers, orders, payments) from PostgreSQL to Snowflake using Hevo Data, then transformed the raw data into a customer analytics model using dbt Core. The architecture keeps extraction and loading separate from transformation — Hevo owns the EL, dbt owns the T.

Layer	Tool	Output
Source	PostgreSQL 15 (Docker)	customers, orders, payments tables with sample data
Extract + Load	Hevo Data	Raw tables in Snowflake PUBLIC schema (POSTGRES_RAW_*)
Transform	dbt Core	STAGING views → MARTS.customers materialized table
Validation	dbt test + SQL queries	All schema + custom tests pass; row counts verified

2. Assumptions

- Hevo performs Extract + Load only. All transformation logic lives in dbt — no Hevo drag-and-drop transformations were used.
- Prefixes replicated tables with POSTGRES_ (e.g., POSTGRES_RAW_CUSTOMERS) for better readability. dbt source definitions use this naming.
- Hevo adds __hevo__marked_deleted to each row. All staging models filter WHERE __hevo__marked_deleted = false to handle CDC soft deletes.
- Customers with no orders are valid records. The final model uses LEFT JOIN + COALESCE to keep all 100 customers — 31 have zero orders and appear with number_of_orders = 0, customer_lifetime_value = 0.
- Snowflake created in AWS Singapore region. The Thailand region caused Hevo Partner Connect to fail with a connection error. Recreating the Snowflake account in Singapore resolved it.
- All credentials live in ~/.dbt/profiles.yml (never committed). The repo contains only profiles.yml.example with placeholder values.

3. Steps Taken

PostgreSQL (Source)

- Ran PostgreSQL 15 in Docker with WAL logical replication (wal_level=logical). Loaded 100 customers, 99 orders, 99 payments.
- Created hevo_replication user, granted SELECT on all tables, created a replication slot (pgoutput plugin) and publication (hevo_publication FOR ALL TABLES).

```

arpan@Arpans-MacBook-Pro init % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
a0cc4cb4afdb   postgres:15   "docker-entrypoint.s..." 19 minutes ago Up 19 minutes 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp   hevo_postgres

arpan@Arpans-MacBook-Pro init % docker exec -t hevo_postgres psql -U hevo_user -d hevo_source -c "\dt;"
List of relations
Schema | Name          | Type  | Owner
-----|-----|-----|-----
public | raw_customers | table | hevo_user
public | raw_orders   | table | hevo_user
public | raw_payments | table | hevo_user
(3 rows)

```

Hevo Pipeline

- Created source: PostgreSQL via ngrok TCP tunnel (local machine exposed to Hevo cloud at 0.tcp.ngrok.io:<port>).
- Set destination: Snowflake (Singapore region). Ingestion mode: Logical Replication (CDC).
Note: Three tables synced. Table name altered with prefix postgres_.

Hevo Pipeline:

#2

PostgreSQL Source

PostgreSQL • Ingests every 30 minutes

→

Snowflake Partner Connect

Snowflake • Loads every 1 hour

ACTIVE

Pipeline Activity

2h 12h 24h ...

Logical Replication

Position: Seq No: 0/198A148

Historical Load (100%)

3/3 Objects Ingested

INGESTING

Last ingested: 3 hours ago

Asia (Singapore) TRIAL EXPIRES IN 14 DAYS Schedule A Demo Arpan Banerjee

#2

PostgreSQL Source

PostgreSQL • Ingests every 30

Host: 0.tcp.in.ngrok.io

Port: 16011

User: hevo_replication

Merge Tables: true

Database: hevo_source

Pipeline Mode: WAL

Load historical data: true

Replication Slot: hevo_slot_1777397012143

Output plugin: test_decoding

ACTIVE

Pipeline Activity

2h 12h 24h ...

Logical Replication

Position: Seq No: 0/198A148

INGESTION

0 epm

TRANSFORM

0 epm

LOAD

0 epm

Replicate JSON Fields: Replicate JSON fields to JSON columns

Include New Objects: true

Destination Prefix: postgres_

Objects (0 of 3)

Events Ingested

1-3 of 3

raw_payments

Historical Load Ingested

Position: id=113

0

ACTIVE

raw_customers

Historical Load Ingested

Position: id=100

0

ACTIVE

raw_orders

Historical Load Ingested

Position: id=99

0

ACTIVE

Snowflake Tables created::

My Workspace > Untitled.sql

ACCOUNTADMIN • COMPUTE_WH (X-Small)

1

2

3

4

5

6

SELECT table_name,

table_schema,

table_owner

FROM

pc_hevodata_db.information_schema.tables

WHERE

table_schema = 'PUBLIC'

AND table_owner = 'PC_HEVODATA_ROLE';

Results (just now)

Table Chart

4 rows 491m

TABLE_NAME	TABLE_SCHEMA	TABLE_OWNER
POSTGRES_RAW_PAYMENTS	PUBLIC	PC_HEVODATA_ROLE
POSTGRES_RAW_ORDERS	PUBLIC	PC_HEVODATA_ROLE
POSTGRES_RAW_CUSTOMERS	PUBLIC	PC_HEVODATA_ROLE

dbt Transformation

NOTE: Wanted to use Hevo DBT core, but which is not available in my case, so dbt practice is performed in local.

- Staging layer: three views (stg_customers, stg_orders, stg_payments) that rename columns and filter soft-deleted rows. Land in STAGING schema via generate_schema_name macro override.
- Marts layer: customers materialized table — aggregates first_order, most_recent_order, number_of_orders, customer_lifetime_value per customer using LEFT JOINs.

```
dbt debug    # verified Snowflake connection
dbt run      # built all 4 models (3 views + 1 table)
dbt test     # all tests passed
```

dbt debug::

```
[(base) arpan@Arpans-MacBook-Pro HevoData-dbt-assesment % dbt debug --profiles-dir ~/.dbt
09:39:27 Running with dbt=1.11.8
09:39:27 dbt version: 1.11.8
09:39:27 python version: 3.12.2
09:39:27 python path: /opt/anaconda3/bin/python
09:39:27 os info: macOS-15.3.1-x86_64-i386-64bit
09:39:28 Using profiles dir at /Users/arpan/.dbt
09:39:28 Using profiles.yml file at /Users/arpan/.dbt/profiles.yml
09:39:28 Using dbt_project.yml file at /Users/arpan/HevoData-dbt-assesment/dbt_project.yml
09:39:28 adapter type: snowflake
09:39:28 adapter version: 1.11.4
09:39:28 Configuration:
09:39:28   profiles.yml file [OK found and valid]
09:39:28   dbt_project.yml file [OK found and valid]
09:39:28 Required dependencies:
09:39:28   - git [OK found]

09:39:28 Connection:
09:39:28   account: MXFMYMR-WF63023
09:39:28   user: jblarpanyahoo
09:39:28   database: PC_HEVODATA_DB
09:39:28   warehouse: COMPUTE_WH
09:39:28   role: ACCOUNTADMIN
09:39:28   schema: PUBLIC
09:39:28   authenticator: None
09:39:28   oauth_client_id: None
09:39:28   query_tag: None
09:39:28   client_session_keep_alive: False
09:39:28   host: None
09:39:28   port: None
09:39:28   proxy_host: None
09:39:28   proxy_port: None
09:39:28   protocol: None
09:39:28   connect_retries: 1
09:39:28   connect_timeout: None
09:39:28   retry_on_database_errors: False
09:39:28   retry_all: False
09:39:28   insecure_mode: False
09:39:28   reuse_connections: True
09:39:28   s3_stage_vpce_dns_name: None
09:39:28   platform_detection_timeout_seconds: 0.0
09:39:28 Registered adapter: snowflake=1.11.4
09:39:29 Connection test: [OK connection ok]
```

dbt run::

```

(base) arpan@Arpans-MacBook-Pro HevoData-dbt-assestment % dbt run --profiles-dir ~/.dbt
09:40:03 Running with dbt=1.11.8
09:40:05 Registered adapter: snowflake=1.11.4
09:40:05 Unable to do partial parsing because saved manifest not found. Starting full parse.
09:40:06 [WARNING][MissingArgumentsPropertyInGenericTestDeprecation]: Deprecated
functionality
Found top-level arguments to test `accepted_values` defined on
'hevo_source_postgres_raw_orders' in package 'HevoData_dbt_assessment'
(models/staging/sources.yml). Arguments to generic tests should be nested under
the `arguments` property.
09:40:06 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 2 unused configuration paths:
- models.hevo_assessment.staging
- models.hevo_assessment.marts
09:40:06 Found 4 models, 20 data tests, 3 sources, 524 macros
09:40:06
09:40:06 Concurrency: 4 threads (target='dev')
09:40:06
09:40:08 1 of 4 START sql view model PUBLIC.stg_customers ..... [RUN]
09:40:08 2 of 4 START sql view model PUBLIC.stg_orders ..... [RUN]
09:40:08 3 of 4 START sql view model PUBLIC.stg_payments ..... [RUN]
09:40:08 1 of 4 OK created sql view model PUBLIC.stg_customers ..... [SUCCESS 1 in 0.48s]
09:40:08 2 of 4 OK created sql view model PUBLIC.stg_orders ..... [SUCCESS 1 in 0.73s]
09:40:10 3 of 4 OK created sql view model PUBLIC.stg_payments ..... [SUCCESS 1 in 1.97s]
09:40:10 4 of 4 START sql table model PUBLIC.customers ..... [RUN]
09:40:12 4 of 4 OK created sql table model PUBLIC.customers ..... [SUCCESS 100 in 2.22s]
09:40:13
09:40:13 Finished running 1 table model, 3 view models in 0 hours 0 minutes and 6.60 seconds (6.60s).
09:40:13
09:40:13 Completed successfully
09:40:13

```

dbt test output:

```
(base) arpan@Arpans-MacBook-Pro HevoData-dbt-assement % dbt test --profiles-dir ~/.dbt
10:24:42 Running with dbt=1.11.8
10:24:44 Registered adapter: snowflake=1.11.4
10:24:44 Found 4 models, 20 data tests, 3 sources, 524 macros
10:24:44
10:24:44 Concurrency: 4 threads (target='dev')
10:24:44
10:24:45 1 of 20 START test assert_first_order_before_recent ..... [RUN]
10:24:45 2 of 20 START test assert_positive_lifetime_value ..... [RUN]
10:24:45 3 of 20 START test not_null_customers_customer_id ..... [RUN]
10:24:45 4 of 20 START test not_null_customers_customer_lifetime_value ..... [RUN]
10:24:46 2 of 20 PASS assert_positive_lifetime_value ..... [PASS in 0.67s]
10:24:46 4 of 20 PASS not_null_customers_customer_lifetime_value ..... [PASS in 0.68s]
10:24:46 3 of 20 PASS not_null_customers_customer_id ..... [PASS in 0.68s]
10:24:46 5 of 20 START test not_null_customers_first_name ..... [RUN]
10:24:46 6 of 20 START test not_null_customers_last_name ..... [RUN]
10:24:46 7 of 20 START test not_null_customers_number_of_orders ..... [RUN]
10:24:46 1 of 20 PASS assert_first_order_before_recent ..... [PASS in 0.78s]
10:24:46 8 of 20 START test source_accepted_values_hevo_source_postgres_raw_orders_status_completed_returned_return_pending_placed_shipped [RUN]
10:24:46 5 of 20 PASS not_null_customers_first_name ..... [PASS in 0.16s]
10:24:46 9 of 20 START test source_accepted_values_hevo_source_postgres_raw_payments_payment_method_credit_card_coupon_bank_transfer_gift_card [RUN]
10:24:46 6 of 20 PASS not_null_customers_last_name ..... [PASS in 0.16s]
10:24:46 10 of 20 START test source_not_null_hevo_source_postgres_raw_customers_first_name [RUN]
10:24:46 7 of 20 PASS not_null_customers_number_of_orders ..... [PASS in 0.17s]
10:24:46 11 of 20 START test source_not_null_hevo_source_postgres_raw_customers_id ..... [RUN]
10:24:46 8 of 20 PASS source_accepted_values_hevo_source_postgres_raw_orders_status_completed_returned_return_pending_placed_shipped [PASS in 0.24s]
10:24:46 12 of 20 START test source_not_null_hevo_source_postgres_raw_customers_last_name [RUN]
10:24:46 11 of 20 PASS source_not_null_hevo_source_postgres_raw_customers_id ..... [PASS in 0.16s]
10:24:46 10 of 20 PASS source_not_null_hevo_source_postgres_raw_orders_status ..... [PASS in 0.17s]
10:24:46 13 of 20 START test source_not_null_hevo_source_postgres_raw_orders_id ..... [RUN]
10:24:46 14 of 20 START test source_not_null_hevo_source_postgres_raw_orders_status ..... [RUN]
10:24:46 9 of 20 PASS source_accepted_values_hevo_source_postgres_raw_payments_payment_method_credit_card_coupon_bank_transfer_gift_card [PASS in 0.24s]
10:24:46 15 of 20 START test source_not_null_hevo_source_postgres_raw_orders_user_id ..... [RUN]
10:24:46 12 of 20 PASS source_not_null_hevo_source_postgres_raw_customers_last_name ..... [PASS in 0.17s]
10:24:46 13 of 20 PASS source_not_null_hevo_source_postgres_raw_orders_id ..... [PASS in 0.16s]
10:24:46 14 of 20 PASS source_not_null_hevo_source_postgres_raw_orders_status ..... [PASS in 0.16s]
10:24:46 16 of 20 START test source_not_null_hevo_source_postgres_raw_payments_id ..... [RUN]
10:24:46 17 of 20 START test source_unique_hevo_source_postgres_raw_customers_id ..... [RUN]
10:24:46 18 of 20 START test source_unique_hevo_source_postgres_raw_orders_id ..... [RUN]
10:24:46 15 of 20 PASS source_not_null_hevo_source_postgres_raw_orders_user_id ..... [PASS in 0.20s]
10:24:46 19 of 20 START test source_unique_hevo_source_postgres_raw_payments_id ..... [RUN]
10:24:47 16 of 20 PASS source_not_null_hevo_source_postgres_raw_payments_id ..... [PASS in 0.18s]
10:24:47 18 of 20 PASS source_unique_hevo_source_postgres_raw_orders_id ..... [PASS in 0.17s]
10:24:47 20 of 20 START test unique_customers_customer_id ..... [RUN]
10:24:47 17 of 20 PASS source_unique_hevo_source_postgres_raw_customers_id ..... [PASS in 0.28s]
10:24:47 19 of 20 PASS source_unique_hevo_source_postgres_raw_payments_id ..... [PASS in 0.20s]
10:24:47 20 of 20 PASS unique_customers_customer_id ..... [PASS in 0.16s]
10:24:48
10:24:48 Finished running 20 data tests in 0 hours 0 minutes and 3.64 seconds (3.64s).
10:24:48
10:24:48 Completed successfully
```

Snowflake MARTS.customers SELECT * LIMIT 10 ::

Home

SQL gsj.sql

SQL Untitled.sql

+

My Workspace > Untitled.sql

▶ ▼

ACCOUNTADMIN - COMPUTE_WH (X-Small)

Choose database

```
1 SELECT *
2 FROM pc_hevodata_db.marts.customers
3 ⚡ | %I to generate
```

Results (just now)

Table

Chart

🔍 🗒 100 rows ⓘ 867ms

	# CUSTOMER_ID	A FIRST_NAME	A LAST_NAME	📅 FIRST_ORDER	📅 MOST_RECENT_ORDER	# NUMBER_OF_ORDERS
		Adam 3.0% Shirley 3.0% +77 more	R. 13.0% H. 11.0% +17 more			
1	1	Michael	P.	2018-01-01	2018-02-10	
2	2	Shawn	M.	2018-01-11	2018-01-11	
3	3	Kathleen	P.	2018-01-02	2018-03-11	
4	4	Jimmy	C.	null	null	
5	5	Katherine	R.	null	null	
6	6	Carol	D.	2018-02-10	2018-03-10	

4. Challenges & How They Were Resolved

Challenge	Root Cause	Resolution
Hevo Partner Connect failed ("Oops something went wrong")	Snowflake account was in Thailand AWS region — not supported by Hevo	Recreated Snowflake account in AWS Singapore region. Partner Connect succeeded.
pg_hba.conf Error 1023 (replication login rejected)	Docker PostgreSQL image does not include replication rules in pg_hba.conf by default	Added 00_pg_hba.sh init script that appends host replication rules before SQL scripts run
Init scripts skipped on restart (\dt showed no tables)	Docker volume from a previous run persisted — PostgreSQL skips init/ if data directory exists	Ran docker-compose down -v to wipe volume, then docker-compose up -d
Staging models landed in PUBLIC not STAGING schema	dbt appends target schema prefix by default (PUBLIC_STAGING)	Added generate_schema_name macro override + +schema: staging in dbt_project.yml

5. Validation Queries & Outputs

Row count verification

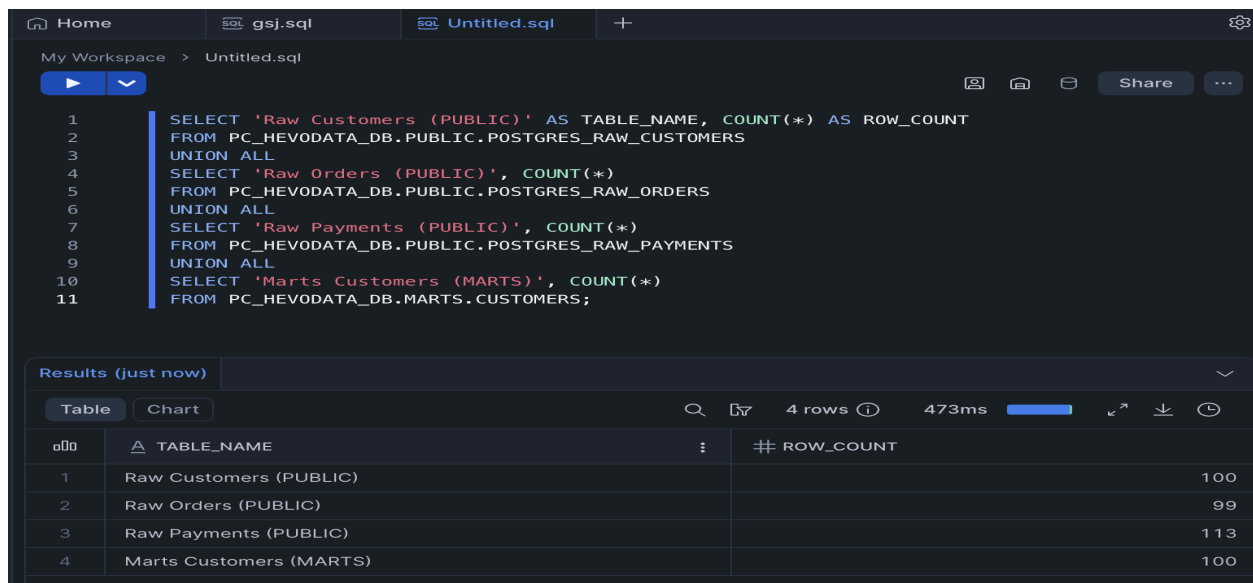
```
-- 1. Row count for Raw Customers (Source)
SELECT COUNT(*)
FROM PC_HEVODATA_DB.PUBLIC.POSTGRES_RAW_CUSTOMERS; -- Expected: 100

-- 2. Row count for Raw Orders (Source)
SELECT COUNT(*)
FROM PC_HEVODATA_DB.PUBLIC.POSTGRES_RAW_ORDERS; -- Expected: 99

-- 3. Row count for Raw Payments (Source)
SELECT COUNT(*)
FROM PC_HEVODATA_DB.PUBLIC.POSTGRES_RAW_PAYMENTS; -- Expected: 113

-- 4. Row count for Final Analytics Table (Marts)
SELECT COUNT(*)
FROM PC_HEVODATA_DB.MARTS.CUSTOMERS; -- Expected: 100
```

Snowflake query results showing row counts::



The screenshot shows a Snowflake SQL editor interface. The top bar includes 'Home', 'gsj.sql', and 'Untitled.sql'. The main workspace shows a SQL query with line numbers 1 through 11. The query is a UNION ALL of four SELECT statements, each counting rows from a different table. Below the editor, the 'Results (just now)' section is visible, showing a table with 4 rows and 2 columns: 'TABLE_NAME' and 'ROW_COUNT'. The results are as follows:

TABLE_NAME	ROW_COUNT
Raw Customers (PUBLIC)	100
Raw Orders (PUBLIC)	99
Raw Payments (PUBLIC)	113
Marts Customers (MARTS)	100

Customers with no orders (LEFT JOIN validation)

```
SELECT COUNT(*)
FROM PC_HEVODATA_DB.MARTS.CUSTOMERS
WHERE NUMBER_OF_ORDERS = 0; -- Expected result: 38
```

Query result: 38 customers with 0 orders::

The screenshot shows a SQL IDE interface with a query editor and a results pane. The query editor contains the following SQL code:

```
1 SELECT COUNT(*)
2 FROM PC_HEVODATA_DB.MARTS.CUSTOMERS
3 WHERE NUMBER_OF_ORDERS = 0; -- Expected result: 38
4
5
6 --I to generate
```

The results pane shows the following table:

#	COUNT(*)
1	38

Top customers by lifetime value

```
SELECT
    CUSTOMER_ID,
    FIRST_NAME,
    LAST_NAME,
    NUMBER_OF_ORDERS,
    CUSTOMER_LIFETIME_VALUE
FROM PC_HEVODATA_DB.MARTS.CUSTOMERS
ORDER BY CUSTOMER_LIFETIME_VALUE DESC
LIMIT 5;
```

Query result: top 5 customers by CLV::

The screenshot shows a SQL IDE interface with a query editor and a results pane. The query editor contains the following SQL code:

```
1 SELECT
2     CUSTOMER_ID,
3     FIRST_NAME,
4     LAST_NAME,
5     NUMBER_OF_ORDERS,
6     CUSTOMER_LIFETIME_VALUE
7 FROM PC_HEVODATA_DB.MARTS.CUSTOMERS
8 ORDER BY CUSTOMER_LIFETIME_VALUE DESC
9 LIMIT 5;
10
11
12
```

The results pane shows the following table:

#	CUSTOMER_ID	FIRST_NAME	LAST_NAME	NUMBER_OF_ORDERS	CUSTOMER_LIFETIME_VALUE
1	51	Howard	R.	3	9900
2	3	Kathleen	P.	3	6500
3	46	Norma	C.	2	6400
4	30	Christina	W.	2	5700
5	54	Rose	M.	5	5700

dbt test results

Test	Model / Column	Result
unique	customer_id (Marts and Source)	PASS
not_null	customer_id, first_name, last_name	PASS
not_null	order_id, customer_id, status	PASS
accepted_values	status (5 values)	PASS
accepted_values	payment_method (4 values)	PASS
assert_positive_lifetime_value	MARTS.customers	PASS
assert_first_order_before_recent	MARTS.customers	PASS

6. Business Questions the customers Model Can Answer Now

- Who are our highest-value customers?
- What is total revenue across all customers?
- How many customers have never ordered?
- Average orders per customer?
- Most loyal customers by order count?
- Revenue distribution across customers?
- Customer order history between two dates?
- Average revenue per ordering customer?

7. Set Up and Run command for the project(High level)

Prerequisites: Docker Desktop, Python 3.9+, ngrok account, Hevo account, Snowflake account (Singapore region).

Step 1 — Start PostgreSQL

```
cd postgres-setup && docker-compose up -d
```

Step 2 — Start ngrok tunnel

```
ngrok tcp 5432 # note the public host:port for Hevo
```

Step 3 — Create Hevo pipeline

Source: PostgreSQL → use ngrok host:port, database: hevo_source, user: hevo_replication. Destination: Snowflake.
Mode: Logical Replication.

Step 4 — Set up dbt

```
pip install dbt-snowflake
# Create ~/.dbt/profiles.yml - see profiles.yml.example in repo root
cd <directory> && dbt debug
```

Step 5 — Run and test

```
dbt run # builds STAGING views + MARTS.customers table
dbt test # runs all schema + custom tests
```

Note: No credentials are committed to the repository. profiles.yml.example contains placeholder values only.